

Simulador DEVS de una Bomba de Infusión Intravenosa

Trabajo Práctico Integrador

Materia: Simulación

Universidad Nacional de Río Cuarto

Autores:

Hernán Jara
Jhonatan Calle

2026

Índice

1. Introducción	3
1.1. Contexto	3
1.2. Objetivo	3
1.3. Alcance	3
1.4. Estructura del documento	4
2. Descripción del Sistema	4
2.1. Visión general	4
2.2. Componentes del sistema	4
2.2.1. Generador de Órdenes Médicas	4
2.2.2. Generador de Fin de Bolsa	5
2.2.3. Generador de Confirmación de Enfermero	5
2.2.4. Sensor de Flujo	5
2.2.5. Actuador de la Bomba	5
2.2.6. Módulo de Alarmas	5
2.2.7. Controlador de Bomba	5
2.3. Modelo acoplado	6
2.4. Parámetros del sistema	7
3. Especificación Formal DEVS	7
3.1. Generador de Órdenes Médicas	8
3.2. Sensor de Flujo	8
3.3. Actuador de la Bomba	10
3.4. Generador de Confirmación de Enfermero	11
3.5. Generador de Fin de Bolsa	12
3.6. Módulo de Alarmas	12
3.7. Controlador de Bomba	14
3.8. Modelo Acoplado	17
4. Implementación en PyPDEVS e infraestructura de validación	18
4.1. Arquitectura de la implementación	18
4.2. Sistema de logging	18
4.3. Verificador de propiedades (P1–P10)	18
4.4. Reporte de resultados	19
4.5. Sistema de gráficos	19
4.6. Integración en <code>main.py</code>	20
5. Escenarios de simulación y resultados	20
5.1. Escenarios definidos	20
5.2. Escenarios determinísticos (1–7)	20
5.3. Escenarios estocásticos (8)	21
5.4. Escenario 9 – Violación deliberada de seguridad	21
5.5. Resultados de verificación	22
5.6. Análisis de resultados	22
5.7. Reporte de métricas	23
5.8. Análisis visual	23

6. Mejoras propuestas	24
6.1. Rate limiting en ajustes de caudal	24
6.2. Watchdog de sensor	25
7. Conclusiones	26

1. Introducción

1.1. Contexto

Una bomba de infusión intravenosa es un dispositivo médico que suministra fluidos, medicamentos o nutrientes de forma controlada al torrente sanguíneo del paciente. Su correcto funcionamiento es crítico para la seguridad del paciente: un caudal incorrecto puede provocar desde una dosificación insuficiente hasta una sobredosis letal.

La simulación basada en el formalismo DEVS (*Discrete Event System Specification*) permite modelar el comportamiento discreto del sistema de control de la bomba, verificar propiedades de seguridad y liveness, y analizar el impacto de fallas en los componentes antes de la implementación física.

1.2. Objetivo

El objetivo de este trabajo es:

- Modelar el sistema de control de una bomba de infusión intravenosa simplificada utilizando el formalismo DEVS.
- Implementar el modelo en Python utilizando el motor de simulación PyPDEVS.
- Definir y verificar un conjunto de propiedades de safety, liveness y temporalidad sobre el comportamiento del sistema.
- Evaluar el sistema frente a distintos escenarios de funcionamiento normal y de falla, incluyendo desvíos de caudal, fin de bolsa, alarmas críticas y una violación deliberada de seguridad.
- Generar gráficos y reportes cuantitativos que permitan analizar el comportamiento del sistema.
- Proponer mejoras para reducir riesgos en el controlador.

1.3. Alcance

El sistema modelado comprende siete modelos atómicos:

`GeneradorOrdenes`, `SensorFlujo`, `ActuadorBomba`, `ControladorBomba`, `ModuloAlarmas`, `GeneradorConfirmacion` y `GeneradorFinBolsa`, interconectados en un modelo acoplado.

Se definieron 8 escenarios determinísticos y estocásticos que cubren operación normal, cambios de orden, desvíos de caudal, fin de bolsa y alarmas críticas, más un escenario adicional (escenario 9) que introduce una violación deliberada de seguridad para demostrar la capacidad del verificador de detectar comportamientos inseguros.

La verificación consta de 10 propiedades (P1–P10) que abarcan safety, liveness y restricciones temporales. Adicionalmente, se genera un reporte de resultados con métricas cuantitativas (alarmas generadas, tiempos de respuesta, detenciones preventivas y porcentaje de tiempo con infusión correcta) y gráficos de caudal, estado de la bomba y timeline de eventos.

1.4. Estructura del documento

El resto del documento se organiza de la siguiente manera:

Sección 2: describe el sistema y sus componentes.

Sección 3: presenta la especificación formal DEVS de cada modelo atómico y del modelo acoplado.

Sección 4: detalla la implementación en PyPDEVS y la infraestructura de validación.

Sección 5: presenta los escenarios de simulación, los resultados obtenidos y el análisis visual.

Sección 6: propone mejoras para reducir riesgos en el controlador.

Sección 7: expone las conclusiones del trabajo.

2. Descripción del Sistema

2.1. Visión general

El sistema modela el comportamiento de una bomba de infusión intravenosa simplificada, encargada de administrar una droga a un paciente respetando una dosis objetivo indicada por una orden médica. El sistema completo se compone de siete modelos atómicos DEVS que se comunican mediante puertos de entrada y salida, formando un modelo acoplado que reproduce tanto el funcionamiento normal del equipo como sus mecanismos de seguridad ante fallas.

El comportamiento se centra en tres ejes:

1. la administración del caudal indicado dentro de los tiempos de respuesta exigidos;
2. la detección y escalado de desvíos sostenidos en el caudal real respecto del objetivo;
3. la reacción ante el agotamiento de la bolsa de medicación.

Todo ello acompañado de un esquema de alarmas que notifica al personal médico y exige su intervención en los casos más críticos.

2.2. Componentes del sistema

A continuación se describen los siete modelos atómicos que componen el sistema.

2.2.1. Generador de Órdenes Médicas

`GeneradorOrdenes` es un modelo autónomo (sin puertos de entrada) que produce periódicamente eventos `ordenMedica` indicando el caudal objetivo (entre 0 y 200 ml/h) que la bomba debe administrar. Una orden con caudal igual a cero indica la detención de la infusión. El generador puede operar en modo determinístico (caudal e interarribo fijos) o estocástico (caudal según distribución Normal, interarribo según distribución Exponencial).

2.2.2. Generador de Fin de Bolsa

`GeneradorFinBolsa` es un modelo autónomo que genera el evento `finBolsa` una única vez cuando la bolsa de medicación está próxima a agotarse. Puede configurarse con un tiempo fijo determinístico o con una distribución Normal para simular variabilidad en el agotamiento del reservorio.

2.2.3. Generador de Confirmación de Enfermero

`GeneradorConfirmacion` es un modelo autónomo que genera periódicamente el evento `confirmacionEnfermero`, simulando el reconocimiento manual de alarmas por parte del personal médico. Opera con un límite máximo de confirmaciones; puede configurarse con un tiempo fijo o con una distribución Log-Normal. La confirmación permite salir del estado de alarma crítica y reanudar la operación.

2.2.4. Sensor de Flujo

`SensorFlujo` mide e informa el caudal real administrado cada 1 segundo exacto. Recibe del actuador el valor físico real (`caudalActual`) y lo incorpora en su próxima medición. Modela además una deriva temporal: cuanto más tiempo pasa sin una actualización del actuador, mayor es la posible discrepancia entre el valor reportado y el valor base, según la tasa `TASA_DERIVA_SENSOR = 2%` del caudal por segundo. Opcionalmente puede agregar ruido gaussiano a la medición.

2.2.5. Actuador de la Bomba

`ActuadorBomba` representa el mecanismo físico que administra la medicación. Recibe del controlador las órdenes `ajustarCaudal` y `detenerBomba`, y aplica el cambio físico tras una latencia fija de 0.5 segundos, notificando luego el nuevo caudal real al sensor mediante `caudalActual`. Puede configurarse con un `factor_falla` que reduce el caudal real respecto del comandado, simulando desperfectos mecánicos (por ejemplo, `factor_falla = 0.80` produce un desvío permanente del 20 %).

2.2.6. Módulo de Alarmas

`ModuloAlarmas` gestiona las notificaciones hacia el personal médico a través del puerto `alarma` (con valores `baja`, `media`, `critica`). Tras notificar una alarma crítica, espera 30 segundos por una confirmación del enfermero; si no llega, repite la notificación cada 10 segundos de forma indefinida hasta ser confirmada. Las alarmas `baja` y `media` son de notificación única. El módulo incluye un guarda que ignora eventos `CRITICA` adicionales mientras ya está manejando una, preservando el primer timer de 30 segundos.

2.2.7. Controlador de Bomba

`ControladorBomba` es el componente central del sistema. Recibe las cuatro entradas externas (`ordenMedica`, `sensorFlujo`, `finBolsa`, `confirmacionEnfermero`) y decide las acciones de control: ajustar el caudal, detener la bomba, o emitir una alarma. Mantiene internamente la máquina de estados de la infusión con las siguientes fases:

- `OCIOSO` — bomba detenida, sin infusión activa.

- **INFUNDIENDO** — infusión en curso, caudal objetivo activo.
- **ALERTA_BOLSA** — fin de bolsa detectado, se permite hasta 60 segundos adicionales de infusión.
- **ALARMA_MEDIA** — desvío sostenido por más de 5 segundos; primera advertencia.
- **ALARMA_CRITICA** — según el criterio propio del equipo (dos episodios de alarma media sin corrección); se detiene la bomba y se exige confirmación.
- **PARADA_POR_BOLSA** — tiempo máximo tras fin de bolsa agotado; la bomba se detiene automáticamente.

El controlador gestiona temporizadores independientes para el desvío de caudal (`t_desvio`) y el fin de bolsa (`t_bolsa`), ambos manejados de forma concurrente mediante la función `_sigma_min()` que devuelve el mínimo de los timers activos. Al recibir una orden médica, programa una demora de 2 segundos antes de emitir `ajustarCaudal`. Cuando detecta el primer desvío, emite un ajuste inmediato (`sigma = 0.0`) e inicia un timer de 5 segundos. En desvío sostenido, el sigma resultante combina el tiempo restante de los timers activos mediante `min(sigma_restante, _sigma_min(t_desvio, t_bolsa))`, preservando el timer pendiente de la orden médica.

El controlador incluye además un guarda configurable (`violar_seguridad`) que permite ignorar las lecturas del sensor durante la fase **ALARMA_CRITICA** y reinicia la fase a **INFUNDIENDO**. Esto se utiliza en el escenario 9 para demostrar la detección de violaciones de seguridad.

2.3. Modelo acoplado

Los siete modelos atómicos se integran en el modelo acoplado **SistemaBomba**, que define las conexiones internas y los puertos de salida observables desde el exterior. Las conexiones son las siguientes:

Con **GeneradorOrdenes** enviando órdenes al controlador:

`ordenMedica` → **ControladorBomba**

El controlador comanda al actuador:

`ajustarCaudal` → **ActuadorBomba**, `detenerBomba` → **ActuadorBomba**

El actuador informa al sensor:

`caudalActual` → **SensorFlujo**

El sensor retroalimenta al controlador:

`sensorFlujo` → **ControladorBomba**

El generador de fin de bolsa notifica al controlador:

`finBolsa` → **ControladorBomba**

La confirmación del enfermero llega tanto al controlador como al módulo de alarmas:

`confirmacionEnfermero` → **ControladorBomba**

`confirmacionEnfermero` $\rightarrow$ `ModuloAlarmas`

Finalmente, el controlador envía alarmas al módulo de alarmas:

`alarma` $\rightarrow$ `ModuloAlarmas`

Los puertos de salida externos del modelo acoplado son:

- `caudalActual` — caudal físico real (del `ActuadorBomba`).
- `registrarEvento` — log de auditoría (del `ControladorBomba`).
- `notificacionAlarma` — mensajes de alarma (del `ModuloAlarmas`).

2.4. Parámetros del sistema

La `config.py` define las constantes del sistema que se listan en la Tabla 1.

Cuadro 1: Parámetros del sistema

Parámetro	Valor	Descripción
PERIODO_SENSOR	1.0 s	Período de muestreo del sensor
LATENCIA_ACTUADOR	0.5 s	Latencia física del actuador
DESVIO_MAX_PERMITIDO	10 %	Desvío máximo tolerado
TIEMPO_MAX_DESVIO	5.0 s	Tiempo máx. con desvío antes de alarma
TIEMPO_CONF_CRITICA	30.0 s	Ventana para confirmar alarma crítica
PERIODO_REP_CRITICA	10.0 s	Período de repetición de alarma crítica
TIEMPO_MAX_FIN_BOLSA	60.0 s	Tiempo máx. tras fin de bolsa
CAUDAL_MIN	0.0 ml/h	Caudal mínimo
CAUDAL_MAX	200.0 ml/h	Caudal máximo
TIEMPO_INICIO_INFUSION	3.0 s	Tiempo máx. para iniciar infusión tras orden
TASA_DERIVA_SENSOR	2 %/s	Deriva del sensor por segundo sin corrección

Además, los parámetros estocásticos incluyen la media y desviación estándar del fin de bolsa (Normal), los parámetros μ y σ de la confirmación (Log-Normal), y la media y desvío del caudal e interarribo de las órdenes (Normal y Exponencial respectivamente). Todos los generadores estocásticos utilizan la semilla 42 para garantizar reproducibilidad.

3. Especificación Formal DEVS

A continuación se presenta la especificación formal de cada modelo atómico utilizando el lenguaje CML-DEVS. Cada especificación define el conjunto de estados (S), los puertos de entrada (X) y salida (Y), la función de transición interna (δ_{int}), la función de transición externa (δ_{ext}), la función de salida (λ) y la función de avance temporal (ta).

3.1. Generador de Órdenes Médicas

El modelo **GeneradorOrdenes** es un modelo atómico autónomo (sin entradas) que genera periódicamente eventos `ordenMedica`, indicando el caudal objetivo que debe administrar la bomba. Las órdenes pueden generarse de manera determinística (caudal y tiempo entre órdenes fijos) o estocástica (caudal con distribución gaussiana, interarribo con distribución exponencial) según los escenarios definidos en `config.py`.

```
atomic GeneradorOrdenes is <Y, S,  $\delta_{int}$ ,  $\lambda$ , ta> where
S is
    caudalProximo :  $\mathbb{R}$ ;      -- caudal objetivo (0 a 200 ml/h)
    interarribo    : Time;    -- tiempo hasta la siguiente orden
     $\sigma$           : Time;    -- variable de avance de tiempo
end S
Y is
    ordenMedica :  $\mathbb{R}$ ;      -- puerto de salida: valor del caudal
end Y
 $\delta_{int}$ (caudalProximo, interarribo,  $\sigma$ ) is
    -- Determinístico: valores fijos del config
    -- Estocástico: caudal ~ Gauss(media, desvío), interarribo ~ Exp(media)
    caudalProximo = next_caudal_logic();
    interarribo    = next_interarribo_logic();
     $\sigma$  = interarribo;
end  $\delta_{int}$ 
 $\lambda$ (caudalProximo, interarribo,  $\sigma$ ) is
    ordenMedica = caudalProximo;
end  $\lambda$ 
ta(caudalProximo, interarribo,  $\sigma$ ) is
     $\sigma$ ;
end ta
end atomic
```

Figura 1: Especificación CML-DEVS: Generador de Órdenes Médicas

Análisis del modelo

1. **Modos de generación:** En modo determinístico, los valores `caudal_fijo` e `interarribo_fijo` provienen del `config.py`. En modo estocástico, el caudal se muestrea de una distribución Gaussiana acotada a $[0, 200]$ ml/h y el tiempo entre órdenes de una distribución Exponencial.
2. **Estado inicial:** Se inicializa con el primer caudal y el primer interarribo calculados en el constructor, de modo que la primera emisión ocurre exactamente al tiempo `interarribo_fijo` (o su equivalente estocástico).

3.2. Sensor de Flujo

El modelo **SensorFlujo** reporta periódicamente el caudal real cada `PERIODO_SENSOR` segundos (por defecto 1s). Al recibir `caudalActual` del actuador, incorpora ese valor y modela una deriva física proporcional al tiempo transcurrido sin actualización, más un ruido gaussiano opcional.

```
atomic SensorFlujo is <X, Y, S,  $\delta_{int}$ ,  $\delta_{ext}$ ,  $\lambda$ , ta> where
```

```

S is
  caudalMedido      : ℝ;      -- valor de caudal a reportar
  caudalActuador    : ℝ;      -- último valor recibido del actuador
  tiempoActuador    : Time;   -- tiempo desde última actualización
  σ                 : Time;   -- controla el período de muestreo
end S
X is
  caudalActual : ℝ; -- caudal físico recibido del actuador
end X
Y is
  sensorFlujo : ℝ; -- medición enviada al Controlador
end Y
δext((caudalMedido, caudalActuador, tiempoActuador, σ),
    e, (puerto, valor)) is
  σ      = max(0.0, σ - e);
  caudalActuador = valor;
  caudalMedido   = valor;
  tiempoActuador = 0.0;
  if (puerto = caudalActual)
end δext
δint(caudalMedido, caudalActuador, tiempoActuador, σ) is
  -- deriva = caudalActuador * TASA_DERIVA * tiempoActuador
  -- ruido ~ Gauss(0, ruido_std) si ruido_std > 0
  caudalMedido = max(0.0, caudalActuador + deriva + ruido);
  tiempoActuador = tiempoActuador + σ;
  σ = PERIODO_SENSOR;
end δint
λ(caudalMedido, caudalActuador, tiempoActuador, σ) is
  sensorFlujo = caudalMedido;
end λ
ta(caudalMedido, caudalActuador, tiempoActuador, σ) is
  σ;
end ta
end atomic

```

Figura 2: Especificación CML-DEVS: Sensor de Flujo

Análisis del modelo

1. **Frecuencia de muestreo:** El sensor reporta el caudal cada PERIODO_SENSOR segundos. σ se resetea a PERIODO_SENSOR en cada transición interna.
2. **Incorporación del valor del actuador:** Al recibir caudalActual, el modelo actualiza caudalActuador y caudalMedido de forma inmediata, resetea tiempoActuador a 0,0 y avanza el cronograma con $\sigma = \max(0, \sigma - e)$.
3. **Deriva y ruido:** La medición incorpora una deriva proporcional al caudal base y al tiempo sin actualización ($TASA_DERIVA_SENSOR \times tiempoActuador$), más un ruido gaussiano de desviación estándar ruido_std (configurable por escenario; 0 lo desactiva).

3.3. Actuador de la Bomba

El modelo **ActuadorBomba** representa la interfaz física de la bomba: recibe comandos lógicos del controlador y, tras una latencia de `LATENCIA_ACTUADOR` segundos (0.5s), actualiza el caudal real suministrado al paciente. Incorpora un `factor_falla` configurable que simula degradación mecánica.

```
atomic ActuadorBomba is
  <X, Y, S,  $\delta_{int}$ ,  $\delta_{ext}$ ,  $\lambda$ , ta> where
S is
  caudalFisico    :  $\mathbb{R}$ ;          -- caudal que sale de la bomba
  caudalObjetivo  :  $\mathbb{R}$ ;          -- caudal a alcanzar tras latencia
   $\sigma$            : Time;         -- latencia física
end S
X is
  ajustarCaudal  :  $\mathbb{R}$ ;          -- nuevo valor desde el controlador
  detenerBomba   : {signal};      -- parada de emergencia
end X
Y is
  caudalActual   :  $\mathbb{R}$ ;          -- notifica cambio al sensor
end Y
 $\delta_{ext}((\text{caudalFisico}, \text{caudalObjetivo}, \sigma), e, (\text{puerto}, \text{valor}))$  is
  defcases
    caudalObjetivo = valor;     $\sigma$  = LATENCIA_ACTUADOR;
    if (puerto = ajustarCaudal)
      caudalObjetivo = 0.0;     $\sigma$  = LATENCIA_ACTUADOR;
      if (puerto = detenerBomba)
    end defcases
  end  $\delta_{ext}$ 
 $\delta_{int}(\text{caudalFisico}, \text{caudalObjetivo}, \sigma)$  is
  caudalFisico = caudalObjetivo * factor_falla;
   $\sigma$          = infinity;
end  $\delta_{int}$ 
 $\lambda(\text{caudalFisico}, \text{caudalObjetivo}, \sigma)$  is
  -- Aplica factor_falla al emitir
  caudalActual = round(caudalObjetivo * factor_falla, 3);
end  $\lambda$ 
ta(caudalFisico, caudalObjetivo,  $\sigma$ ) is
   $\sigma$ ;
end ta
end atomic
```

Figura 3: Especificación CML-DEVS: Actuador de la Bomba

Análisis del modelo

1. **Latencia configurable:** La constante `LATENCIA_ACTUADOR` se importa de `config.py` (valor por defecto: 0.5s). Ambas ramas de δ_{ext} la aplican, garantizando que todo cambio de caudal, incluyendo la parada de emergencia, experimenta el mismo retardo físico.
2. **Factor de falla:** Parámetro configurable por escenario. Un valor de 1,0 indica funcionamiento nominal; valores inferiores simulan degradación mecánica. Se apli-

ca tanto en λ (notificación al sensor) como en δ_{int} (actualización del estado físico interno).

3. **Estado pasivo:** Cuando no hay órdenes pendientes, $\sigma = \infty$. El actuador mantiene el caudal actual indefinidamente hasta que el controlador envíe un nuevo evento.

3.4. Generador de Confirmación de Enfermero

El modelo **GeneradorConfirmacion** es un modelo atómico autónomo que genera eventos `confirmacionEnfermero` un número limitado de veces (`max_confirmaciones`), simulando el reconocimiento manual de alarmas. Soporta modo determinístico (intervalo fijo) y estocástico (distribución log-normal).

```
atomic GeneradorConfirmacion is
  <Y, S,  $\delta_{int}$ ,  $\lambda$ , ta> where
S is
   $\sigma$  : Time; -- tiempo hasta siguiente confirmación
  confirmaciones :  $\mathbb{N}$ ; -- cantidad emitidas
end S
Y is
  confirmacionEnfermero : {signal};
end Y
 $\delta_{int}(\sigma, confirmaciones)$  is
  confirmaciones = confirmaciones + 1;
  -- Determ.: sigma = tiempo_fijo
  -- Esto.: sigma ~ LogNormal(mu, sigma_ln)
  -- Si >= max_confirmaciones: sigma = infinity
   $\sigma$  = next_sigma(confirmaciones);
end  $\delta_{int}$ 
 $\lambda(\sigma, confirmaciones)$  is
  confirmacionEnfermero = signal;
end  $\lambda$ 
ta( $\sigma, confirmaciones$ ) is
   $\sigma$ ;
end ta
end atomic
```

Figura 4: Especificación CML-DEVS: Generador de Confirmación

Análisis del modelo

1. **Estado extendido:** El estado incluye `confirmaciones`, un contador que se incrementa en cada δ_{int} y permite al modelo detenerse automáticamente al alcanzar `max_confirmaciones`.
2. **Modos de operación:** En modo *determinístico*, σ se fija en `tiempo_fijo`. En modo *estocástico*, $\sigma \sim \text{LogNormal}(\mu, \sigma_{\ln})$, modelando la variabilidad real del tiempo de respuesta del personal médico.

3.5. Generador de Fin de Bolsa

El modelo **GeneradorFinBolsa** es un modelo atómico autónomo que genera el evento `finBolsa` una única vez cuando la bolsa de medicación se agota. Soporta modo determinístico (tiempo fijo) y estocástico (distribución gaussiana).

```
atomic GeneradorFinBolsa is
  <Y, S,  $\delta_{int}$ ,  $\lambda$ , ta> where
S is
   $\sigma$  : Time;    -- tiempo hasta fin de bolsa
  emitido : {True, False}; -- bandera: ya se emitió
end S
Y is
  finBolsa : {signal};
end Y
 $\delta_{int}(\sigma, emitido)$  is
  emitido = True;
   $\sigma$  = infinity;
end  $\delta_{int}$ 
 $\lambda(\sigma, emitido)$  is
  finBolsa = signal;
end  $\lambda$ 
ta( $\sigma, emitido$ ) is
   $\sigma$ ;
end ta
end atomic
```

Figura 5: Especificación CML-DEVS: Generador de Fin de Bolsa

Análisis del modelo

1. **Emisión única:** Tras emitir `finBolsa`, δ_{int} pone $\sigma = \infty$ y `emitido = True`. El modelo no vuelve a activarse.
2. **Modos de operación:** En modo determinístico, $\sigma = \text{tiempo_fijo}$. En modo estocástico, $\sigma \sim \mathcal{N}(\text{media}, \text{desvío_std})$, acotado inferiormente a 0,1 para garantizar positividad.

3.6. Módulo de Alarmas

El modelo **ModuloAlarmas** gestiona las notificaciones hacia el personal médico, emitiendo alertas y manteniendo un ciclo de retransmisión para las alarmas críticas no confirmadas. Los tiempos de espera y repetición (`TIEMPO_CONF_CRITICA` y `PERIODO_REP_CRITICA`) se importan de `config.py`.

```
atomic ModuloAlarmas is
  <X, Y, S,  $\delta_{int}$ ,  $\delta_{ext}$ ,  $\lambda$ , ta> where
S is
  tipoAlarma : {BAJA, MEDIA, CRITICA, NINGUNA};
  fase       : {OCIOSO, NOTIFICANDO, ESPERANDO_CONF, REPETIENDO};
   $\sigma$        : Time;
end S
X is
```

```

    alarma                : {baja, media, critica};
    confirmacionEnfermero : {signal};
end X
Y is
    notificacionAlarma : Text;
end Y
 $\delta_{ext}((\text{tipoAlarma}, \text{fase}, \sigma), e, (\text{puerto}, \text{valor}))$  is
     $\sigma = \max(0.0, \sigma - e)$ ;
    defcases
        tipoAlarma = NINGUNA; fase = OCIOSO;  $\sigma = \text{infinity}$ ;
        if (puerto = confirmacionEnfermero)
            -- CRITICA no es reemplazada; MEDIA no por BAJA
            tipoAlarma = valor; fase = NOTIFICANDO;  $\sigma = 0.0$ ;
            if (puerto = alarma  $\wedge$  tipoAlarma  $\neq$  CRITICA
                 $\wedge$  not(tipoAlarma = MEDIA  $\wedge$  valor = baja))
            end if
        end defcases
end  $\delta_{ext}$ 
 $\delta_{int}(\text{tipoAlarma}, \text{fase}, \sigma)$  is
    defcases
        fase = ESPERANDO_CONF;  $\sigma = \text{TIEMPO\_CONF\_CRITICA}$ ;
        if (tipoAlarma = CRITICA  $\wedge$  fase = NOTIFICANDO)
            fase = REPETIENDO;  $\sigma = \text{PERIODO\_REP\_CRITICA}$ ;
            if (tipoAlarma = CRITICA  $\wedge$ 
                (fase = ESPERANDO_CONF  $\vee$  fase = REPETIENDO))
                tipoAlarma = NINGUNA; fase = OCIOSO;  $\sigma = \text{infinity}$ ;
                if (tipoAlarma = MEDIA  $\vee$  tipoAlarma = BAJA)
                end if
            end if
        end defcases
end  $\delta_{int}$ 
 $\lambda(\text{tipoAlarma}, \text{fase}, \sigma)$  is
    notificacionAlarma = "ALERTA: " + tipoAlarma;
end  $\lambda$ 
ta(tipoAlarma, fase,  $\sigma$ ) is
     $\sigma$ ;
end ta
end atomic

```

Figura 6: Especificación CML-DEVS: Módulo de Alarmas

Análisis del modelo

1. **Prioridad de alarmas:** Una alarma CRITICA activa no puede ser reemplazada por ninguna otra, y una alarma MEDIA activa no es reemplazada por una BAJA. La confirmación del enfermero siempre tiene prioridad.
2. **Lógica de repetición crítica:** Tras la primera notificación, δ_{int} transiciona a ESPERANDO_CONF con $\sigma = 30$ s. Si no llega confirmación, pasa al estado REPETIENDO con ciclos de $\sigma = 10$ s.
3. **Alarmas no críticas:** MEDIA y BAJA se notifican una única vez; δ_{int} resetea a OCIOSO con $\sigma = \infty$.

3.7. Controlador de Bomba

El modelo **ControladorBomba** es el componente central del sistema. Recibe órdenes médicas, monitorea el caudal real, detecta desvíos y condiciones de riesgo, y coordina las acciones sobre el actuador y el módulo de alarmas. Incorpora la fase PARADA_POR_BOLSA y el parámetro violar_seguridad para escenarios de verificación formal.

```

atomic ControladorBomba is
  <X, Y, S,  $\delta_{int}$ ,  $\delta_{ext}$ ,  $\lambda$ , ta> where
S is
  caudalObjetivo :  $\mathbb{R}$ ;
  caudalReal      :  $\mathbb{R}$ ;
  fase : {OCIOSO, INFUNDIENDO, ALERTA_BOLSA,
          ALARMA_MEDIA, ALARMA_CRITICA, PARADA_POR_BOLSA};
  contadorMedia  :  $\mathbb{N}$ ;
   $\sigma$           : Time;
  t_desvio       : Time;
  t_bolsa        : Time;
end S
X is
  ordenMedica      :  $\mathbb{R}$ ;      -- caudal objetivo
  sensorFlujo      :  $\mathbb{R}$ ;      -- caudal real medido
  finBolsa         : {signal}; -- bolsa casi vacía
  confirmacionEnfermero : {signal};
end X
Y is
  ajustarCaudal    :  $\mathbb{R}$ ;      -- comando al actuador
  detenerBomba     : {signal}; -- parada de emergencia
  alarma          : {b, m, c}; -- alertas al módulo
  registrarEvento  : Text;     -- registro de auditoría
end Y
 $\delta_{ext}((\text{caudalObjetivo}, \text{caudalReal}, \text{fase}, \text{contadorMedia},$ 
   $\sigma, \text{t\_desvio}, \text{t\_bolsa}), e, (\text{puerto}, \text{valor}))$  is
  -- Actualizar timers
  if (t_desvio  $\neq$  infinity): t_desvio = max(0.0, t_desvio - e);
  if (t_bolsa  $\neq$  infinity): t_bolsa = max(0.0, t_bolsa - e);
  sigma_restante = max(0.0,  $\sigma$  - e);
  defcases
    caudalObjetivo = clamp(valor); fase = INFUNDIENDO;  $\sigma$  = 2.0;
    if (puerto = ordenMedica  $\wedge$  valor > 0.0)
  caudalObjetivo = 0.0; fase = OCIOSO;  $\sigma$  = 0.0;
    if (puerto = ordenMedica  $\wedge$  valor  $\leq$  0.0)
  -- Sensor: primer desvío
  caudalReal = valor; t_desvio = TIEMPO_MAX_DESVIO;  $\sigma$  = 0.0;
    if (puerto = sensorFlujo  $\wedge$ 
  fase  $\neq$  OCIOSO  $\wedge$  fase  $\neq$  ALARMA_CRITICA
     $\wedge$  t_desvio = infinity  $\wedge$ 
  abs(valor - caudalObjetivo) > DESVIO_MAX * caudalObjetivo)
  -- Sensor: desvío continuo
  caudalReal = valor;
   $\sigma$  = min(sigma_restante,
    sigma_min(t_desvio, t_bolsa));

```

```

        if (puerto = sensorFlujo ∧
            fase ≠ OCIOSO ∧ fase ≠ ALARMA_CRITICA
            ∧ t_desvio ≠ infinity ∧
            abs(valor - caudalObjetivo) > DESVIO_MAX * caudalObjetivo)
-- Sensor: desvío corregido
caudalReal = valor; t_desvio = infinity; contadorMedia = 0;
σ = sigma_restante;
        if (puerto = sensorFlujo ∧
            abs(valor - caudalObjetivo) ≤ DESVIO_MAX * caudalObjetivo)
fase = ALERTA_BOLSA; t_bolsa = TIEMPO_MAX_FIN_BOLSA;
σ = 0.0;
        if (puerto = finBolsa)
fase = OCIOSO; σ = 0.0; contadorMedia = 0;
        if (puerto = confirmacionEnfermero
            ∧ fase = ALARMA_CRITICA)
end defcases
end δext
δint(caudalObjetivo, caudalReal, fase, contadorMedia,
    σ, t_desvio, t_bolsa) is
-- Decrementar timers por sigma expirado
if (t_bolsa ≠ infinity): t_bolsa = max(0.0, t_bolsa - σ);
if (t_desvio ≠ infinity): t_desvio = max(0.0, t_desvio - σ);
defcases
-- 1. Timeout fin de bolsa (máxima prioridad)
caudalObjetivo = 0.0; fase = PARADA_POR_BOLSA;
t_bolsa = infinity; σ = 0.0;
    if (t_bolsa ≤ 0.0)
-- 2. Timeout de desvío: escalar alarma
contadorMedia = contadorMedia + 1; fase = ALARMA_MEDIA;
t_desvio = infinity; σ = 0.0;
        if (t_desvio ≤ 0.0 ∧ contadorMedia < 2)
contadorMedia = 3; fase = ALARMA_CRITICA;
t_desvio = infinity; σ = 0.0;
        if (t_desvio ≤ 0.0 ∧ contadorMedia ≥ 2)
-- 3. PARADA_POR_BOLSA -> OCIOSO
fase = OCIOSO; σ = infinity;
        if (fase = PARADA_POR_BOLSA)
-- 4. ALARMA_MEDIA -> INFUNDIENDO
fase = INFUNDIENDO; σ = sigma_min(t_desvio, t_bolsa);
        if (fase = ALARMA_MEDIA)
-- 5. ALERTA_BOLSA -> INFUNDIENDO
fase = INFUNDIENDO; σ = sigma_min(t_desvio, t_bolsa);
        if (fase = ALERTA_BOLSA)
-- 6. INFUNDIENDO: ajuste de caudal
σ = sigma_min(t_desvio, t_bolsa);
        if (fase = INFUNDIENDO)
-- 7. ALARMA_CRITICA u OCIOSO -> pasivo
σ = infinity;
        if (fase = ALARMA_CRITICA ∨ fase = OCIOSO)
end defcases
end δint

```



```

λ(caudalObjetivo, caudalReal, fase, contadorMedia,
  σ, t_desvio, t_bolsa) is
defcases
  ajustarCaudal = clamp(caudalObjetivo);
  registrarEvento = "Ajustando caudal";
  if (fase = INFUNDIENDO)
    alarma = baja;
    registrarEvento = "Fin de bolsa -- alarma BAJA";
    if (fase = ALERTA_BOLSA)
      alarma = media;
      registrarEvento = "Desvío sostenido -- alarma MEDIA";
      if (fase = ALARMA_MEDIA)
        alarma = critica; detenerBomba = signal;
        registrarEvento = "Alarma CRITICA -- bomba detenida";
        if (fase = ALARMA_CRITICA)
          detenerBomba = signal;
          registrarEvento = "Tiempo fin de bolsa -- bomba detenida";
          if (fase = PARADA_POR_BOLSA)
            detenerBomba = signal;
            registrarEvento = "Orden caudal=0 -- bomba detenida";
            if (fase = OCIOSO ∧ caudalObjetivo = 0.0)
              end defcases
end λ
ta(caudalObjetivo, caudalReal, fase, contadorMedia,
  σ, t_desvio, t_bolsa) is
  σ;
end ta
end atomic

```

Figura 7: Especificación CML-DEVS: Controlador de Bomba

Análisis del modelo

1. **Fase PARADA_POR_BOLSA:** Cuando el temporizador t_bolsa expira, el controlador transiciona a **PARADA_POR_BOLSA** en lugar de directamente a **OCIOSO**. Esto permite que λ emita **detenerBomba** con un mensaje de auditoría específico, diferenciándolo de una parada por orden médica con caudal cero.
2. **Gestión de temporizadores concurrentes:** En δ_{ext} se descuenta el tiempo transcurrido e de los timers activos usando $\max(0, t - e)$. La función auxiliar $\sigma_min(t_desvio, t_bolsa)$ retorna el menor de los timers con valor finito, o ∞ si ambos están apagados.
3. **Criterio de escalado (decisión del equipo):** El equipo definió como criterio propio — según el requisito de la consigna que delega en cada grupo la definición del escalado — que, tras emitir una **ALARMA_MEDIA** sin que el desvío se corrija, el controlador contabiliza la falla en **contadorMedia**. Al alcanzar dos desvíos sostenidos consecutivos (que generan dos alarmas medias), el controlador escala a **ALARMA_CRITICA** y, por el puerto **detenerBomba**, detiene físicamente el motor, cumpliendo así con la propiedad de seguridad (safety) del sistema.
4. **Parámetro violar_seguridad:** Permite forzar que el controlador ignore el estado

ALARMA_CRITICA y reanude el control, habilitando escenarios de verificación formal que demuestran el incumplimiento de propiedades de seguridad.

5. **Demora inicial de 2 segundos:** Al recibir una nueva orden médica con caudal positivo, el controlador establece $\sigma = 2,0$ antes de emitir `ajustarCaudal`, modelando el tiempo de procesamiento clínico previo a la actuación física.

3.8. Modelo Acoplado

El modelo acoplado **SistemaBomba** integra los siete modelos atómicos descritos anteriormente. La Figura 8 muestra el diagrama de conexiones, y la Tabla 2 enumera los enlaces entre puertos.

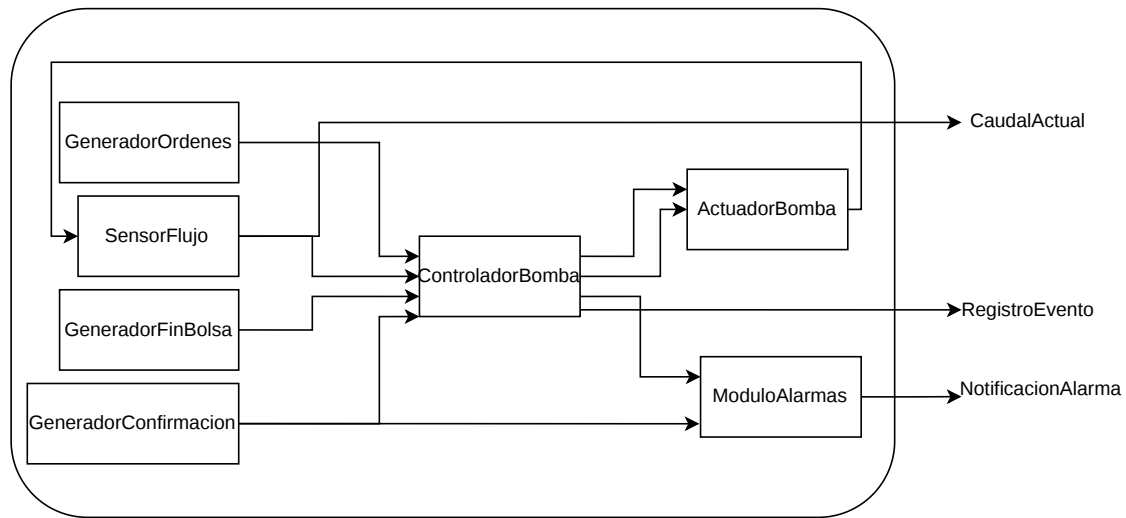


Figura 8: Modelo Acoplado del Sistema de Bomba de Infusión

Cuadro 2: Conexiones internas del modelo acoplado

Origen	Destino
GeneradorOrdenes.ordenMedica	ControladorBomba.ordenMedica
ControladorBomba.ajustarCaudal	ActuadorBomba.ajustarCaudal
ControladorBomba.detenerBomba	ActuadorBomba.detenerBomba
ActuadorBomba.caudalActual	SensorFlujo.caudalActual
SensorFlujo.sensorFlujo	ControladorBomba.sensorFlujo
GeneradorFinBolsa.finBolsa	ControladorBomba.finBolsa
GeneradorConfirmacion.confirmacion	ControladorBomba.confirmacion
GeneradorConfirmacion.confirmacion	ModuloAlarmas.confirmacion
ControladorBomba.alarma	ModuloAlarmas.alarma

Los puertos de salida externos del modelo acoplado son:

- `caudalActual` — caudal físico real (del ActuadorBomba).

- `registrarEvento` — log de auditoría (del `ControladorBomba`).
- `notificacionAlarma` — mensajes de alarma (del `ModuloAlarmas`).

4. Implementación en PyPDEVS e infraestructura de validación

4.1. Arquitectura de la implementación

Cada modelo atómico se implementa como una subclase de `AtomicDEVS` de la biblioteca PyPDEVS. El estado de cada modelo se almacena en un diccionario (`self.state`) y se actualiza en `extTransition()` e `intTransition()` retornando un nuevo diccionario. La función de avance temporal `timeAdvance()` retorna `self.state["sigma"]`.

El modelo acoplado `SistemaBomba` extiende `CoupledDEVS` e importa los siete modelos atómicos, conectándolos mediante `connectPorts()`. Los puertos de salida externos (`caudalActual`, `registrarEvento`, `notificacionAlarma`) permiten observar el comportamiento desde el exterior de la simulación.

La simulación se ejecuta con el motor `Simulator` de PyPDEVS en modo `Classic DEVS`, con los parámetros de configuración definidos en `config.py` y la semilla 42 para los generadores estocásticos. El tiempo de terminación se configura mediante `setTerminationTime()`.

4.2. Sistema de logging

Para capturar los eventos durante la simulación se implementó un `EventLogger` que almacena tuplas (tiempo, puerto, valor) en una lista ordenada. La clase `TracerEventLogger` integra este logger con el sistema de *tracing* de PyPDEVS, reemplazando el uso de `setVerbose()`. El tracer se instala mediante:

```
1 sim.setCustomTracer("logger.event_logger", "TracerEventLogger",
2                     [logger, verbose])
```

El parámetro `verbose` controla si los eventos se imprimen en consola durante la simulación, y el `EventLogger` se pasa por referencia para que el verificador pueda acceder a los eventos una vez finalizada la simulación.

4.3. Verificador de propiedades (P1–P10)

El `VerificadorPropiedades` recorre el `EventLogger` post-simulación y verifica 10 propiedades distribuidas en tres categorías:

Safety (P1–P3)

- **P1 – Caudal cero:** Si la última orden médica es caudal cero, no debe haber `ajustarCaudal > 0` después.
- **P2 – Caudal máximo:** Ningún `ajustarCaudal` supera el límite de 200 ml/h.

- **P3 – Crítica sin confirmación:** Tras una alarma crítica, no debe haber `ajustarCaudal` hasta que llegue `confirmacionEnfermero` o una nueva orden médica.

Liveness (P4–P6)

- **P4 – Orden produce acción:** Toda `ordenMedica` debe generar `ajustarCaudal` o `detenerBomba` dentro de los 3 segundos posteriores.
- **P5 – Crítica se repite:** Toda alarma crítica no confirmada debe retransmitirse eventualmente.
- **P6 – Fin de bolsa detiene:** Todo evento `finBolsa` debe ser seguido de `detenerBomba`.

Temporalidad (P7–P10)

- **P7 – Inicio de infusión:** Toda orden con caudal positivo produce un ajuste en menos de 3 segundos.
- **P8 – Alarma media en 5 segundos:** Un desvío sostenido por más de 5 segundos dispara `alarma = "media"`.
- **P9 – Fin de bolsa en 60 segundos:** La detención por fin de bolsa ocurre dentro de los 60 segundos posteriores.
- **P10 – Crítica se repite cada 10 segundos:** La primera repetición de una crítica no confirmada ocurre a los 30 segundos y las siguientes cada 10 segundos.

Cada propiedad retorna un `ResultadoVerificacion` con el estado (cumplida/fallida), un detalle textual y la evidencia (eventos involucrados). Las propiedades cuyo antecedente nunca se activa se consideran verdaderas por vacuidad.

4.4. Reporte de resultados

La función `generar_reporte()` en `verificacion/reporte_resultados.py` calcula métricas cuantitativas a partir del `EventLogger`:

- **Alarmas generadas:** conteo por tipo (`baja`, `media`, `critica`).
- **Tiempo de respuesta a desvío:** tiempo promedio desde el primer ajuste hasta la alarma media.
- **Tiempo de respuesta a fin de bolsa:** tiempo entre `finBolsa` y `detenerBomba`.
- **Detenciones preventivas:** cantidad total de eventos `detenerBomba`.
- **Porcentaje de infusión correcta:** proporción del tiempo donde el caudal real se encuentra dentro del 10% del objetivo, calculado recorriendo la línea de tiempo combinada de órdenes y caudales reales.

4.5. Sistema de gráficos

La función `graficar_escenario()` en `graficos/graficar_resultados.py` genera una figura PNG con tres subplots:

1. **Caudal objetivo vs caudal real:** tres curvas escalón (`drawstyle='steps-post'`) para órdenes médicas (azul), ajustes de caudal (verde) y caudal real (rojo), más líneas punteadas verticales para alarmas críticas.
2. **Estado de la bomba:** reconstrucción de las fases del controlador (OCIOSO, INFUNDIENDO, ALERTA_BOLSA, MEDIA, CRITICA, PARADA) con bandas de colores de fondo.
3. **Timeline de eventos:** cada tipo de evento en una fila separada con marcadores diferenciados, facilitando la lectura visual de la secuencia temporal.

La figura incluye un bloque de texto al pie con los parámetros del escenario, los resultados de verificación y las métricas del reporte, permitiendo tener toda la información en una sola imagen. La imagen se guarda en `graficos/escenario_{num}_{slug}.png` con 150 dpi.

4.6. Integración en `main.py`

El punto de entrada `main.py` coordina todo el flujo:

1. Selecciona la configuración del escenario según el argumento `--escenario` (1–9).
2. Crea un `EventLogger` y configura el `TracerEventLogger` como tracer personalizado.
3. Instancia `SistemaBomba` y `Simulator`, ejecuta la simulación en modo Classic DEVS.
4. Si se especifica `--verificar` o `--graficos`, ejecuta el `VerificadorPropiedades` y muestra los resultados en consola.
5. Si se especifica `--graficos`, genera el PNG con `graficar_escenario()` incluyendo el reporte de métricas.

5. Escenarios de simulación y resultados

5.1. Escenarios definidos

Se definieron 9 escenarios que cubren operación normal, condiciones de falla y una violación deliberada de seguridad. La Tabla 3 resume los parámetros principales.

5.2. Escenarios determinísticos (1–7)

A continuación se describe cada escenario y su comportamiento esperado.

Escenario 1 – Operación normal. El actuador entrega el caudal exacto de la orden (100 ml/h) cada 20 segundos. No hay desvío, no hay fin de bolsa ni alarmas. El sistema permanece en INFUNDIENDO todo el tiempo.

Escenario 2 – Órdenes estocásticas. El caudal de las órdenes se muestrea de una Gaussiana $\mathcal{N}(100, 15)$ ml/h y el interarribo de una Exponencial con media 10 segundos (semilla 42). El sistema sigue el caudal variable sin desvíos sostenidos.

Cuadro 3: Resumen de escenarios de simulación

N.	Descripción
1	Operación normal: caudal fijo 100 ml/h cada 20 s, sin fallas
2	Órdenes estocásticas (Gaussiana), sin fallas
3	Orden única con caudal = 0 (detención programada)
4	Desvío leve: actuador al 85 % (15 % de desvío), <5 s
5	Desvío grave: actuador al 80 % (20 % de desvío), sostenido, sin confirmación
6	Fin de bolsa determinístico en t=30 s, confirmación en t=50 s
7	Alarma crítica sin confirmación (deriva de sensor)
8	Fin de bolsa estocástico (Normal) con confirmación estocástica (Log-Normal) y ruido en sensor
9	Violación: mismo desvío que escenario 5 pero con <code>violar_seguridad=True</code>

Escenario 3 – Orden caudal cero. Una única orden con caudal 0 a los 5 segundos. El controlador emite `detenerBomba` y queda en `OCIOSO`. Verifica P1.

Escenario 4 – Desvío leve. El actuador opera al 85 % (`factor_falla = 0.85`), generando un desvío del 15 %. La orden única a los 20 segundos provoca un ajuste; el desvío persiste menos de 5 segundos antes de que llegue la siguiente orden, por lo que no se dispara alarma media.

Escenario 5 – Desvío grave. El actuador opera al 80 % (`factor_falla = 0.80`), generando un desvío permanente del 20 %. Sin confirmación del enfermero, el desvío sostenido supera 5 segundos, dispara alarma media, y al acumularse dos episodios de alarma media sin corrección, escala a alarma crítica, deteniendo la bomba.

Escenario 6 – Fin de bolsa. La bolsa se agota en t=30 segundos. El controlador emite alarma baja y permite 60 segundos adicionales de infusión. En t=50 segundos llega la confirmación del enfermero; si no hubiera llegado, la bomba se detendría a los 90 segundos (30 + 60).

Escenario 7 – Alarma crítica sin confirmación. Sin fin de bolsa ni falla en el actuador, la deriva del sensor (`TASA_DERIVA_SENSOR = 2%/s`) genera un desvío que escala a alarma media y luego a crítica. Sin confirmación, la alarma crítica se repite cada 30 segundos y luego cada 10 segundos.

5.3. Escenarios estocásticos (8)

Escenario 8 – Fin de bolsa estocástico. Combina fin de bolsa con distribución Normal ($\mu = 30$, $\sigma = 2$), confirmación Log-Normal ($\mu = 2,5$, $\sigma_{\ln} = 0,8$), y ruido gaussiano en el sensor ($\sigma = 0,5$). Evalúa el comportamiento del sistema bajo variabilidad estadística.

5.4. Escenario 9 – Violación deliberada de seguridad

El escenario 9 replica el escenario 5 (desvío grave) pero con el flag `violar_seguridad=True`. Esto desactiva el guarda que impide que el controlador procese lecturas del sensor

durante la fase `ALARMA_CRITICA`. Como consecuencia, tras la detención por alarma crítica, una nueva lectura del sensor reactiva la infusión, generando un `ajustarCaudal` ilegal. El verificador detecta esta violación en P3:

```
1 [X] P3: Safety - critica sin confirmacion
2   Ajuste antes de reanudacion en t=50.0
```

Las otras 9 propiedades (P1–P10 excepto P3) se cumplen, demostrando que la violación es quirúrgica y no afecta el resto del sistema.

5.5. Resultados de verificación

La Tabla 4 muestra los resultados de las 10 propiedades para cada escenario. La simbología sigue la misma convención de la matriz de cobertura: ✓ para propiedades cumplidas, ✗ para violadas, y — para verdad vacua.

Cuadro 4: Resultados de verificación por escenario

Prop	1	2	3	4	5	6	7	8	9
P1	—	—	✓	—	—	—	—	—	—
P2	✓	✓	✓	✓	✓	✓	✓	✓	✓
P3	—	—	—	—	✓	—	✓	—	✗
P4	✓	✓	✓	✓	✓	✓	✓	✓	✓
P5	—	—	—	—	✓	—	✓	—	✓
P6	—	—	—	—	—	✓	—	✓	—
P7	✓	✓	—	✓	✓	✓	✓	✓	✓
P8	—	—	—	✓	✓	—	✓	—	✓
P9	—	—	—	—	—	✓	—	✓	—
P10	—	—	—	—	✓	—	✓	—	✓

5.6. Análisis de resultados

Los resultados confirman que:

- **Todas las propiedades tienen cobertura:** cada propiedad tiene al menos un escenario donde su antecedente se activa y el consecuente se evalúa realmente. No hay propiedades siempre vacuas.
- **Solo P3 falla en el escenario 9:** la violación deliberada de seguridad demuestra que el verificador detecta correctamente el comportamiento indebido. Las 9 propiedades restantes se cumplen incluso en este escenario, lo que indica que la violación es localizada.
- **P2, P4 y P7 son ubicuas:** se activan en casi todos los escenarios porque no requieren condiciones especiales (ajustes y órdenes existen en prácticamente todos los casos).
- **P6 y P9 son las más específicas:** solo se activan en escenarios con evento de fin de bolsa (6 y 8).

5.7. Reporte de métricas

Para cada escenario se calculan métricas cuantitativas que permiten evaluar el rendimiento del sistema:

- **Alarmas generadas:** el escenario 5 (desvío grave sin confirmación) genera 6 alarmas (2 medias, 4 críticas). El escenario 9 (violación) genera 16 alarmas (2 medias, 14 críticas) porque la violación reinicia el ciclo de desvío repetidamente.
- **Tiempo de respuesta a desvío:** en los escenarios con desvío sostenido (4, 5, 7, 9), la alarma media se dispara aproximadamente 5 segundos después del primer ajuste, cumpliendo el requisito temporal.
- **Detenciones preventivas:** el escenario 5 presenta 4 detenciones y el escenario 9 presenta 14 detenciones, reflejo del reinicio continuo del ciclo de alarma crítica.
- **Porcentaje de infusión correcta:** en los escenarios sin falla (1, 2, 3) el porcentaje es cercano al 100 %. En escenarios con desvío permanente (5, 9) el porcentaje es 0 % porque nunca se alcanza el caudal objetivo. En el escenario 6 (fin de bolsa con confirmación) el porcentaje se mantiene alto porque la infusión se detiene ordenadamente.

5.8. Análisis visual

Los gráficos generados para cada escenario (Figura 9) permiten visualizar simultáneamente:

1. La evolución del caudal objetivo, los ajustes y el caudal real (curvas escalón), con líneas punteadas verticales para alarmas críticas.
2. Las fases del controlador (OCIOSO, INFUNDIENDO, ALERTA_BOLSA, MEDIA, CRITICA, PARADA) con bandas de colores.
3. El timeline de eventos con marcadores diferenciados para cada tipo de evento (órdenes, ajustes, alarmas, confirmaciones, fin de bolsa, detenciones).

Cada PNG incluye al pie los parámetros del escenario, el resultado de la verificación y las métricas del reporte, ofreciendo una visualización completa del comportamiento del sistema.

Escenario 5: Desvío grave (> 5 s)

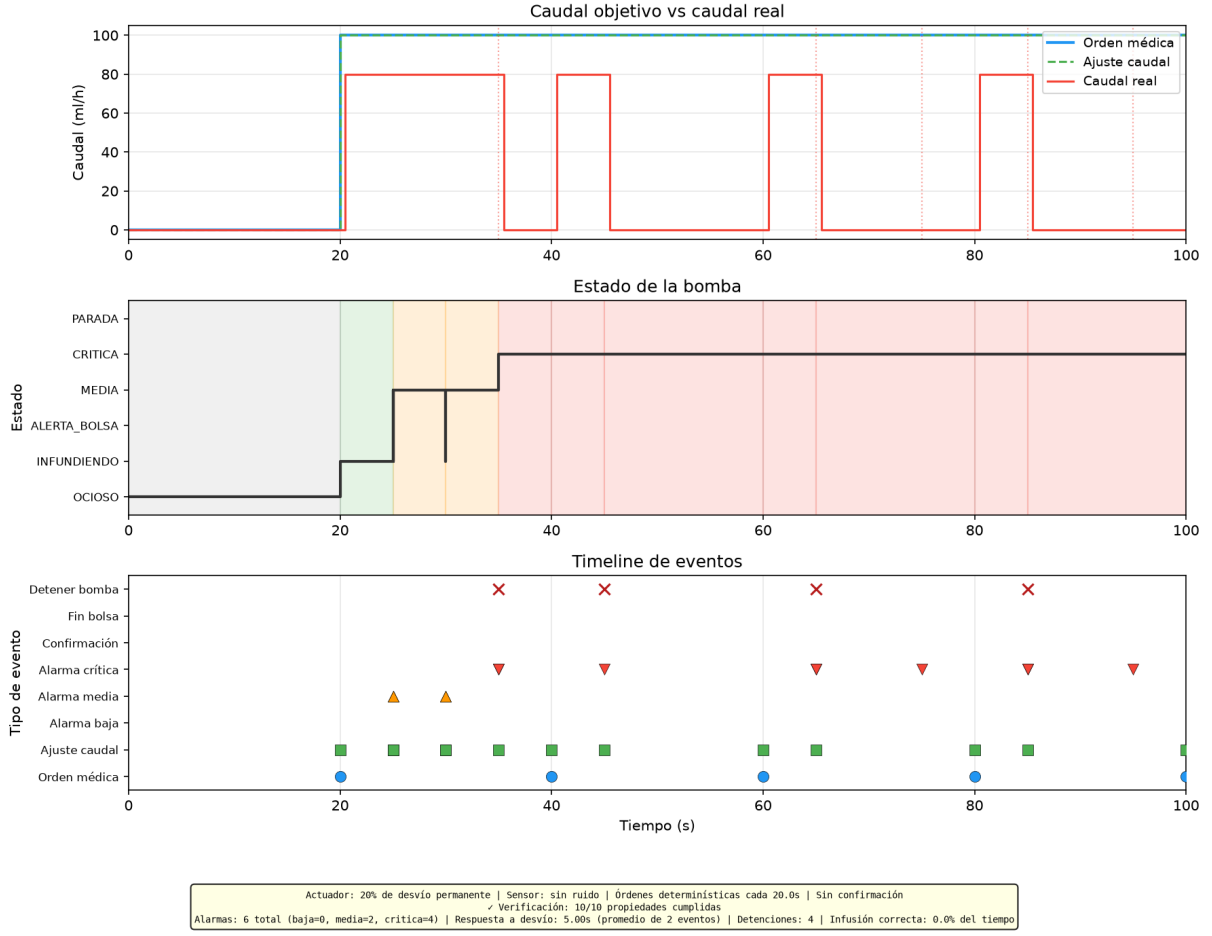


Figura 9: Ejemplo de gráfico generado: escenario 5 (desvío grave)

6. Mejoras propuestas

A partir del análisis de los resultados de simulación y verificación, se identificaron dos áreas de mejora para reducir riesgos en el controlador de la bomba de infusión.

6.1. Rate limiting en ajustes de caudal

Problema identificado. En la implementación actual, cuando el sensor detecta un desvío sostenido, el controlador puede emitir ajustes de caudal en cada ciclo del sensor (cada 1 segundo) mediante la expresión:

$$\sigma = \min(\sigma_{\text{restante}}, \sigma_{\min}(t_{\text{desvío}}, t_{\text{bolsa}}))$$

Cuando $\sigma_{\text{restante}} \approx 1,0$ (próxima transición interna del controlador, que coincide aproximadamente con el período del sensor) y $t_{\text{desvío}} > 0$, el sigma resultante es $\approx 1,0$, generando un ajuste cada segundo. Sin embargo, el actuador tiene una latencia física de 0.5 segundos (**LATENCIA_ACTUADOR**) para aplicar cada cambio. Si el controlador emite un nuevo ajuste antes de que el actuador haya completado el anterior, se producen dos problemas:

1. **Oscilaciones:** el controlador corrige sobre un valor que aún no se ha materializado, pudiendo sobrepasar el objetivo.
2. **Ineficiencia:** el actuador recibe más comandos de los que puede procesar, desperdiciando ciclos de simulación.

Propuesta. Agregar una constante `MIN_INTERVALO_AJUSTE = 0.5` (igual a la latencia del actuador) y modificar la lógica de sigma en desvío sostenido para que el controlador no emita ajustes con una frecuencia mayor a la que el actuador puede responder:

$$\sigma = \min(\max(\sigma_{\text{restante}}, \text{MIN_INTERVALO_AJUSTE}), \sigma_{\min}(t_{\text{desvío}}, t_{\text{bolsa}}))$$

Impacto esperado.

- Menor cantidad de eventos `ajustarCaudal`.
- Curva de caudal real más estable (menos oscilaciones).
- El porcentaje de tiempo con infusión correcta se mantiene o mejora porque los ajustes son más efectivos.

6.2. Watchdog de sensor

Problema identificado. El sistema asume que el sensor de flujo siempre reporta lecturas cada `PERIODO_SENSOR = 1.0` segundos, pero no hay ningún mecanismo que verifique que el sensor está funcionando correctamente. Si el sensor falla (se desconecta, se rompe, o el cable se daña):

1. El controlador nunca recibe eventos `sensorFlujo`.
2. Permanece en la última fase en la que estaba.
3. Si estaba en `INFUNDIENDO` y no hay desvío, sigue infundiendo indefinidamente sin supervisión.
4. Si el caudal real se desvía (obstrucción parcial o falla del actuador), el controlador no lo detecta por falta de mediciones.

Esto constituye un riesgo de seguridad grave: la bomba puede estar entregando un caudal incorrecto sin que el sistema lo sepa.

Propuesta. Agregar un **watchdog timer** al estado del controlador que detecte la ausencia de lecturas del sensor:

- El controlador lleva la cuenta del tiempo desde la última lectura del sensor en `t_ultimoSensor`.
- En cada `intTransition`, incrementa `t_ultimoSensor` por σ .
- Si `t_ultimoSensor > 2 × PERIODO_SENSOR = 2,0` segundos, se considera que el sensor falló y se entra en un estado `FALLA_SENSOR`.
- En `FALLA_SENSOR`, se emite `alarmaCritica`, `detenerBomba`, y se queda pasivo hasta recibir confirmación.

- Al recibir una nueva lectura del sensor (`sensorFlujo` en `extTransition`), se resetea $t_{\text{ultimoSensor}} = 0$ y se sale de `FALLA_SENSOR`.

Escenario de prueba propuesto. Se podría crear un **escenario 10: falla de sensor** donde el `SensorFlujo` deje de emitir después de cierto tiempo. A los 2 segundos sin lecturas, el controlador debe entrar en `FALLA_SENSOR`, emitir alarma crítica y detener la bomba. Se agregaría una propiedad P11: *si el sensor deja de reportar por más de 2 segundos, se debe emitir alarma crítica y detener la bomba.*

7. Conclusiones

En este trabajo se presentó el modelado, la implementación y la verificación de un controlador para bomba de infusión intravenosa utilizando el formalismo DEVS y la herramienta PyPDEVS. El sistema fue especificado mediante siete modelos atómicos DEVS que representan los componentes físicos y lógicos de la bomba, integrados en un modelo acoplado que reproduce el comportamiento del sistema real.

Se diseñaron nueve escenarios de simulación que cubren operación normal, condiciones de falla (desvío de caudal, fin de bolsa, deriva de sensor) y una violación deliberada de seguridad. Para cada escenario se ejecutaron diez verificaciones de propiedades (P1–P10) que abarcan requisitos de seguridad, correcto funcionamiento y temporales. Todas las propiedades se cumplen en los escenarios nominales y de falla; únicamente la propiedad P3 falla en el escenario 9 (violación deliberada), demostrando que el verificador detecta correctamente el comportamiento indebido.

La infraestructura de validación desarrollada — compuesta por un registrador de eventos, un verificador de propiedades, un generador de reportes de métricas y un visualizador gráfico — demostró ser eficaz para evaluar el comportamiento del sistema de forma sistemática. La matriz de cobertura confirma que cada propiedad tiene al menos un escenario donde su antecedente se activa, lo que garantiza que ninguna propiedad es siempre vacua.

El modelado estocástico de órdenes médicas, fin de bolsa, confirmaciones y ruido del sensor permitió evaluar la robustez del sistema bajo condiciones de incertidumbre. La inclusión de semillas fijas ($\text{seed} = 42$) asegura la reproducibilidad de los resultados.

Entre las limitaciones del trabajo se encuentra la ausencia de un mecanismo de watchdog para detectar fallas del sensor y la falta de rate-limiting en los ajustes de caudal, aspectos que quedan como trabajo futuro. Asimismo, la separación del puerto único de alarmas en tres puertos diferenciados (baja, media, crítica) mejoraría la modularidad del modelo.

En conclusión, el enfoque basado en DEVS con verificación formal de propiedades resultó adecuado para modelar y validar un sistema de control de infusión intravenosa, demostrando que es posible detectar automáticamente violaciones de seguridad y garantizar el cumplimiento de requisitos críticos en un sistema de software para salud.